

ProtoViz: Application & Transport Layer Visualizer

An interactive educational web platform demonstrating synchronized protocol flows across L7 (DNS, HTTP/1.1, SMTP, HLS) and L4 (TCP 3-way handshake, Seq/Ack math, state machine, flow & congestion control).

■ **Live Website:** <https://protoboot.up.railway.app/>
■ **GitHub Repository:**
<https://github.com/ayu37071/Application-Layer>

Author: Ayush Chauhan
Stack: Python 3.13, FastAPI,
WebSockets, JS
Testing: 15/15 Pytest Passed (100%)

1. Executive Overview & Objectives

ProtoViz is a web-based educational dashboard designed to demystify computer network protocol interactions by simultaneously presenting **user activity** on the left panel and **in-depth protocol exchanges** on the right panel. Designed across Assignment 1 (Application Layer) and Assignment 2 (Transport Layer), the system provides seamless dual-layer visibility.

Students can perform intuitive high-level actions (visiting a URL, composing an email, or streaming video), and watch the resulting network wire exchanges breakdown progressively in real time. The platform establishes strict **bi-directional cross-layer linking**: inspecting an HTTP GET request automatically identifies its underlying TCP packets, while clicking a transport segment highlights the corresponding application step.

2. System Capabilities & Key Features

A. Application-Layer (L7) Activities

- **Interactive Web Browsing:** Simulates full DNS resolution (RFC 1035 UDP queries to 8.8.8.8) followed by HTTP/1.1 (RFC 9112) GET and 200 OK exchanges with header breakdowns (Host, User-Agent, Content-Type, Content-Length). Includes quick preset buttons (example.com, wiki.org, etc.).
- **Email Protocol Client (SMTP):** Simulates an RFC 5321 mail delivery transaction over Port 25: 220 Service Ready → EHLO → MAIL FROM → RCPT TO → DATA (with multipart/MIME ASCII formatting) → QUIT → 221 Closing.
- **Adaptive Video Streaming (HLS):** Simulates HTTP Live Streaming (RFC 8216) with resolution selection (360p, 720p HD, 1080p FHD). Fetches Master M3U8 Playlist, chunk variant schedule, and downloads consecutive .ts media segments, reflected live in a mock player buffer bar.

B. Transport-Layer (L4) Mechanics (Assignment 2)

- **RFC 793 & RFC 9293 TCP State Engine:** Accurate 10-state finite state machine modeling (CLOSED, LISTEN, SYN-SENT, SYN-RECEIVED, ESTABLISHED, FIN-WAIT-1, FIN-WAIT-2, CLOSE-WAIT, LAST-ACK, TIME-WAIT) displayed in live badges.
- **3-Way Handshake & 4-Way Teardown:** Step-by-step visualization of connection initiation (SYN → SYN-ACK → ACK) with initial sequence number (ISN) negotiation, and clean teardown (FIN-ACK → ACK → FIN-ACK → ACK).
- **Deterministic Seq/Ack Arithmetic:** Exact byte-stream sequence and acknowledgement number progression where SYN/FIN consume 1 phantom byte and data packets consume exactly *payload_length* bytes.

- **Congestion Window (cwnd) & Flow Control:** Visual representation of Slow Start (exponential cwnd doubling) transitioning at ssthresh to Congestion Avoidance (+1 MSS additive increase), alongside advertised receive window flow control.
- **Network Fault & Packet Loss Injection:** Deterministic fault simulation toggling packet drops, triggering Retransmission Timeout (RTO) duplicate sequence number emissions.

C. Interactive Playback & Analysis Dock

- **Synchronized Controls:** Full playback control bar (Auto-play with 1.3s calibrated ticker, Pause, Resume, Next Step, Previous Step, Replay, and Direct Timeline Seeking).
- **Layer-Aware Stepping:** The stepper dynamically adapts to the user's active view: advancing application messages when on L7, and walking through every transport segment (including handshakes and acknowledgments) when on L4.
- **Analysis Dock Shelf:** Dedicated docked panels for *Live Telemetry Stats*, *Congestion Chart*, *TCP vs UDP Architecture Matrix*, *QUIC & HTTP/3 Educational Notes*, and *Fault Controls*.

3. Protocol Simulation Matrix

Protocol	Layer	Standard / RFC	Ports	Key Visualized Fields & Mechanics
DNS	L7 (App)	RFC 1035	53 (UDP)	Query/Response, Transaction ID, QNAME, QTYPE (A/AAAA), TTL, Answer RDATA
HTTP/1.1	L7 (App)	RFC 9112	80 (TCP)	GET request, Status 200 OK, Host, User-Agent, Content-Type, Content-Length
SMTP	L7 (App)	RFC 5321	25 (TCP)	220 Banner, EHLO, MAIL FROM, RCPT TO, DATA, 250 OK, QUIT, 221 Bye
HLS	L7 (App)	RFC 8216	80/443	M3U8 master playlist parsing, video chunk scheduling, buffer level emulation
TCP	L4 (Trans)	RFC 793/9293	Ephemeral	SEQ, ACK, Flags [SYN, ACK, PSH, FIN], 64KB Window, 10-state machine, cwnd
UDP	L4 (Trans)	RFC 768	53 (DNS)	Connectionless datagrams, 8-byte header, zero-handshake overhead demonstration

4. Role of Artificial Intelligence (AI) in Development

This project was architected, developed, and verified utilizing **Google Antigravity (AGY)**, an agentic AI pair-programming system. Rather than generating disconnected boilerplate, AI served as an active co-engineer managing codebase state, running unit tests, and refining interfaces.

■ Architecture & Data Modeling

Designed modular Pydantic v2 schemas (protocol_models.py) ensuring strict type-safety across Python backend and client JSON events. Established clean separation between Application Layer and Transport Layer simulation engines.

■ RFC-Compliant Protocol Engineering

Formulated the mathematical sequence/acknowledgment engine in transport_simulator.py conforming to RFC 793/9293, computing correct relative time offsets, sliding windows, and 10 distinct TCP FSM states.

■ Layer-Aware WebSocket Concurrency

Built an asynchronous WebSocket manager (websocket_manager.py) using Python's asyncio.Lock to govern thread-safe playback tickers, enabling bidirectional pointer alignment between L4 segments and L7 application events.

■ Modern UX/UI Design System

Created a high-density, professional dark-mode interface in CSS/Vanilla JS (style.css, app.js) featuring collapsible inspection shelves, directional packet animations, responsive side-by-side grids, and zero third-party UI framework bloat.

■ Automated Quality Assurance

Synthesized a comprehensive 15-test pytest suite (tests/) validating handshake math, teardown sequences, retransmission edge cases, and WebSocket control messages with 100% test pass rate.

■ Iterative Debugging & Live Deployment

Rapidly identified and fixed environment hurdles (venv path migration, Starlette testclient title matching, viewport overflow constraints) and prepared the production Docker container deployed on Railway.

5. Quality Assurance & Verification (15/15 Passed)

The visualizer maintains **15 automated pytest tests** covering all network protocol flows, state transitions, and server endpoints. All tests pass cleanly in 0.18s.

Test Case Identifier	Module	Verification Scope	Result
test_browsing_flow	test_protocol_simulator	DNS query/response & HTTP/1.1 GET/200 OK step generation	PASSED
test_mail_flow	test_protocol_simulator	SMTP command-response sequence integrity (EHLO to QUIT)	PASSED
test_streaming_flow	test_protocol_simulator	HLS manifest retrieval & multiple TS media chunk downloads	PASSED
test_tcp_handshake	test_transport_simulator	SYN, SYN-ACK, ACK sequence and client/server connection states	PASSED
test_tcp_seq_ack_arithmetic	test_transport_simulator	Math correctness of Seq/Ack across data packets & phantom bytes	PASSED
test_tcp_teardown	test_transport_simulator	4-way termination, FIN-WAIT, CLOSE-WAIT, and TIME-WAIT states	PASSED
test_cross_layer_linking	test_transport_simulator	Bi-directional mapping between L7 events and carrier L4 segments	PASSED
test_network_fault_retransmission	test_transport_simulator	Deterministic drop injection, RTO simulation, duplicate Seq emission	PASSED
test_websocket_transport_sync	test_transport_simulator	Live WebSocket sync, seek, next/prev layer-aware stepping	PASSED

6. Project Access & Repository Links

Target Resource	Clickable Web Link	Description
Live Deployment	https://protoboot.up.railway.app/	Production web application hosted on Railway cloud with HTTPS & WebSockets.
GitHub Repository	https://github.com/ayu37071/Application-Layer	Public Git repository containing all source code, test suites, and documentation.